Laborator 9 AA

1. Calculați factorialul unui număr natural n folosind:

a) un algoritm determinist iterativ

b) un algoritm determinist recursiv (recursivitate pe stivă)

Calculați complexitatea spațială pentru cei doi algoritmi.

Soluție

a)

```
fact1(n)
    r = 1
    for i = 1 --- n
        r = r * i
    return r
```

Se va face un singur apel de funcție, în urma căruia se alocă memorie pentru două variabile (i și r)

$i \leq n \Rightarrow O(\lg n)$

$r \leq n! \Rightarrow O(\lg n!)$

$\Rightarrow$

Complexitatea spațială este $O(\lg n!)$

1) b)

```
fact2(n)
    if n=0
        return 1
    else
        return n * fact2(n-1)
```

Lungimea lanțului de apeluri recursive este n. Deoarece se lucrează cu numere $\leq n$, spațiul ocupat la fiecare apel este $O(\lg n)$ $\Rightarrow$
Complexitatea spațială a funcție fact 2 este $O(n \lg n)$

(2.) Calculați suma elementelor unui vector v de n numere naturale ($v[i] <= u$, orice i) folosind:

a) un algoritm determinist iterativ
b) un algoritm determinist recursiv $n \to n/2, n/2$
c) un algoritm determinist recursiv $n \to 1, n-1$

Calculați complexitatea spațială pentru cei 3 algoritmi.

a)

```
suma1(v,n)
    s=0
    for i=1,...,n
        s=s+v[i]
    return s
```

Se face un singur apel de funcție, în urma căruia se folosește memorie pentru două variabile (s, i)

$i \leq n$ ⇒ complexitatea spațială este $O(\lg n)$

$s \leq n \cdot m$, deoarece $v[i] \leq m$ și $i \leq n$ ⇒ complexitatea spațială este $O(\lg(n \cdot m))$

⇒ Complexitatea spațială a funcției suma1 este $O(\lg(n \cdot m))$

b)
```
suma2 ( v, i, j)
    if i=j
        return v[i]
    else
        return suma2( v, i, (i+j)/2) +
               suma2( v, (i+j)/2+1, j)
```

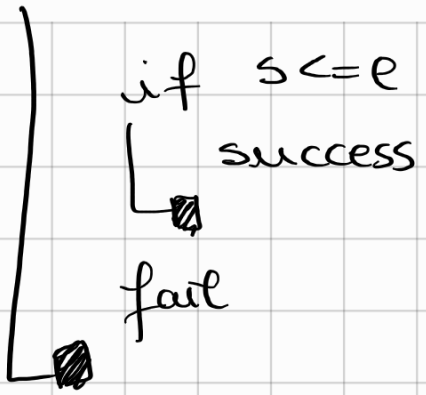
Lungimea maximă a lanțului de recursivitate este $\lg n$. Deoarece $i \leq n$, $j \leq n$, iar valoarea sumei din return este $\leq n \cdot n$ (valoarea maximă posibilă a sumei elementelor din vectorul v) ⇒ complexitatea spațială a unui apel de funcție este $O(\lg n \cdot n)$ ⇒

Complexitatea spațială a funcției suma2 este $O(\lg n \cdot \lg n \cdot n)$

c)
```
suma3(v, i, n)
    if i=n
        return v[n]
    else
        return v[i] + suma3(v, i+1, n)
```

Lungimea lanțului de apeluri recursive este n. Deoarece $i \leq n$, iar valoarea sumei returnate este $\leq m \cdot n$ (valoarea maximă posibilă a sumei elementelor din vectorul v) $\Rightarrow$ complexitatea spațială a unui apel de funcție este $O(\lg nm)$ $\Rightarrow$ complexitatea spațială a funcției suma3 este $O(n \cdot \lg nm)$

3.
```
N_TSP(G, e)
    V' = ∅
    for i = 1, ..., n
        v = choice(V)
        if v in V'
            fail
        V' = V' U {v}
    s = 0
    for i = 1 ... n-1
        s = s + cost(v'[i], v'[i+1])
    s = s + cost(v'[n], v'[1])
```



Rezolvați problema TSP (varianta de decizie) folosind un algoritm nedeterminist cu complexitate temporală polinomială (alg. a fost construit într-un laborator precedent). Impunem restricția cost(arc) <= m, orice arc. Graful G are n noduri. Calculați complexitatea spațială a algoritmului.

Funcția consumă spațiu pentru 4 variabile (v', i, s, v)

- v' va conține maxim n elemente cu indicele $\leq n$ $\Rightarrow$ complexitatea spațială este $O(n \lg n)$
- $i \leq n \Rightarrow$ complexitatea spațială este $O(\lg n)$
- $s \leq n \cdot m$ (suma poate fi maxim $n \cdot m$, deoarece costul $\leq m$, iar în total sunt n muchii) $\Rightarrow$ complexitatea spațială este $O(\lg m \cdot n)$
- $v \leq n$ (v = choice(v)) $\Rightarrow$ v are complexitate spațială $O(\lg n) \Rightarrow$

Complexitatea spațială a funcției este $O(n \lg n + \lg(nm))$